
When Do Language Servers Help Coding Agents?

A Preprint

Ian Barber

July 2026

Abstract

We tested three services a language server offers a coding agent against a text-capable baseline: resolving a symbol to its definition, returning a compact definition span instead of a whole file, and timing a type diagnostic.

Semantic navigation was near-neutral because the model reads the receiver’s type in visible source and self-localizes. It helped only in a constructed regime that withheld that source, and only when the agent was framed to use it: framed lookup resolved every rollout there against 70/75 for text, while merely making the operation available changed nothing. The compact span came from a static definition-span tool, not a live server. It cut total tokens 3.3–4.1x against whole-file retrieval and less against a competent grep-plus-ranged-read interface, but only where it replaced the read; pushing, instructing and self-election did not produce that replacement, and training did. On twelve seeded defects, a checker at the submission boundary moved accepted-correct outcomes from 1/12 to 11/12; the same checker after every edit changed nothing on a channel that fired in one of twelve.

1. What we were investigating

A coding agent already has grep, ranged reads and a shell, so the useful question is not whether a language server can answer questions about code but whether it answers ones the agent cannot cheaply answer for itself, in a cheaper form, or at a better moment.

IF: whether delivering the semantic information changes what the agent finds. A yes is the semantic arm resolving targets the text arm misses; a no is both arms resolving the same tasks at the same cost.

FORM: whether a compact span costs less than reading the file. A yes is total tokens falling at unchanged success; a no is the span arriving as context on top of the reads it was meant to replace.

WHEN: whether the moment a diagnostic arrives changes the outcome. A yes is the same diagnostic changing the accepted result depending only on when it fires; a no is the outcome flat across delivery phases.

Each of the three is a claim that tool builders make. Typed Holes [1] and LSPRAG [2] push types and definitions into the context on the premise that the information is missing, which the first question puts to test; STALL+ [3] reports static-analysis value varying with integration phase, which the third question tests. Neither line reports a counterfactual that already reads code well, so the baseline here is a capable text agent.

2. How it was tested

Every result but one comes from a purpose-built harness: a model drives a fixed loop of six actions — grep, ranged read, whole-file read, line edit, run tests, submit — over a Python workspace, with one semantic operation added or withheld per arm; the exception is an out-of-harness probe in an off-the-shelf bash agent on real repositories. Nothing ran through an MCP server, a skill or an editor plugin.

Dispatch: fifteen tasks where a typed receiver calls one of about ten same-named overrides, one buggy, across a visibility ladder (L0 call-site annotation, L1 test construction site, L2 factory indirection) and a hidden rung withholding `app.py` and the test source, receiver construction redacted. `effic` and `effic_real2`: retrieval-cost tasks, `effic_real2` constructed misuse over vendored real library source (toolz 1.1.0, more-

itertools 11.1.0). navigation-v2: mechanically validated definition-span instances. checker-gate-v3: twelve seeded defect/clean pairs, each defect coherent, visible-passing and held-out-failing with one target-scoped semantic diagnostic, each clean control its validated gold. The other suites, **effic** included, are constructed from scratch: the real-repository scan cleared no admissible candidate (C17).

The efficiency thread’s definition operation is a static AST resolver (`skip_pyrefly=True`, `ast.parse`), not a live language server; live Pyrefly ran only in dispatch goto arms and one 7B run.

The dispatch ladder, navigation-v2, the paired retrieval run and checker-gate-v3 ran at temperature 0, one rollout per cell; the hidden rung and its visible control ran at 0.7 over five seeds per task, 450 rollouts; the whole-file contrast, election and execution-feedback runs at 0.7 over repeated seeds (four behind the 44-attempt cells); the substitution harvest at 0.7 with its held-out retest at 0. Models: Qwen2.5-Coder-7B, Qwen3.5-27B, Qwen3.6-27B locally; Sonnet 4.5, DeepSeek v3.1, GLM-4.6, GPT-5.6 “Luna” via API. Unit of analysis: the task, with task-level bootstrap intervals.

Per-claim configuration and artifacts: [ledger](#), [manifest](#), [protocols](#).

3. Results

3.1 Does delivering the semantic information change what the agent finds?

On the dispatch suite’s annotated rung, grep and ranged reads resolved 15/15, neutral go-to-definition 14/15 and framed 15/15, at matched-success ratios 0.972 and 1.041 (C9). Across rungs L0, L1 and L2, mean input tokens on resolved tasks were 1,436, 1,429 and 1,465 at 14–15/15, goto ratios 0.945–1.065 at every rung including L2, where only the server statically resolves the type. Trajectory counters: about 0.3 greps per task, the receiver’s type read where it appeared, the buggy override opened first.

The hidden rung was run at five seeds per task, temperature 0.7, against a matched visible control (C30):

Arm	Hidden	Paired difference vs grep	Greps	Whole-file reads	Visible
grep_base	70/75	—	1.52	2.17	75/75
defn_avail	71/75	+0.013 [−0.040, +0.067]	1.47	2.04	75/75
defn_prompt	75/75	+0.067 [+0.013, +0.133]	0.36	0.68	75/75

Differences are per-task resolution rates over five seeds, with a 95% task bootstrap interval over the fifteen tasks. Framed lookup improved four tasks (`codec_serialize`, `job_priority`, `resource_cost`, `row_format_row`) and worsened none; making the same operation merely available improved three and worsened two. In the visible control every arm resolved every rollout, so the paired difference there is exactly zero and the effect appears only where the type was withheld. Cost landed on resolution rather than tokens: matched-success input-token ratios were 0.983 framed and 1.005 available. The constructed hidden regime withholds source a real agent could usually open. An earlier single-seed pass at temperature 0 resolved 12/15, 13/15 and 14/15 on the same tasks; its text arm was the one that moved most under sampling, so the temperature-0 figures overstate the gap.

In the typed/erased pilot, typed lookup mechanically reached the exact gold override among 8–15 same-named overrides where erased lookup returned a non-discriminating base, both variants type-clean (C15). Its two-task agent arm passed all 12 task-condition cells at a typed automatic/baseline total-token ratio of 1.037 (0.988–1.093), too wide to establish equivalence; every automatic result was followed by a target-file read, and composed lookup added about six seconds per task (C24).

3.2 Does a compact span cost less than reading the file?

Against whole-file retrieval on the retrieval-cost tasks, compact definition spans cut total tokens 3.35x, 3.49x and 4.12x for Qwen3.6-27B, Sonnet 4.5 and DeepSeek v3.1, at 44/44 resolved in every arm (C1).

Against a grep-plus-ranged-read interface with whole-file fallback, eleven **effic_real2** tasks with Qwen3.5-27B cost 1,602 mean total tokens with text against 1,235 with definitions, a paired ratio of 1.297 (task bootstrap 1.093–1.527), cheaper on 10/11 at 11/11 success in both arms, the one suite and model where the span replaced the read: 0 of 11 definitions were followed by a reread (C27).

Untrained, the 7B elected the definition operation in 1 of 48 rollouts; relabel-trained, in 48 of 48 with no file reads at all, at mean input 3,086 to 687 and resolution 31/48 to 48/48 (C2). Framing raised codenav use on two of three real-repository tasks at one seed (C3, C6).

With `lsp_defn` on in both `effic` arms, the untrained 7B invoked the definition tool 0/24 and the trained arm 24/24, at mean input 2,894 to 689 and success 14/24 to 24/24. The adapter is the election mechanism, and a live-first Pyrefly backend with historical AST fallback reproduces the saving once the policy elects, its per-row backend unrecorded. No arm held the policy fixed while varying the backend, so this is not a measurement of a language-server token effect (C4).

Pushed spans were reread 35/36, and 34/36 with an explicit sufficiency instruction, which removed 2/36 and added 294 mean tokens for the 27B (C31).

Model	Reread, pushed span	Reread, plus sufficiency instruction
Qwen3.6-27B	11/12	12/12
Sonnet 4.5	12/12	12/12
DeepSeek v3.1	12/12	10/12

A separate run with Qwen3.6-27B, GLM-4.6 and GPT-5.6 Luna reproduced the pushed reread at 35/36. Among the instances each model elected, reread was 9/9, 4/9 and 7/12; GLM-4.6 solved only 5–6/12 of these tasks, and reread is counted independently of success (C34).

In a bash agent (mini-swe-agent, Sonnet 4.5) on real repositories, roughly 16 of 18 `codenav defn` calls were followed by a read of the file just resolved, `sympy's str.py` among them at 22 `sed` invocations, on three tasks at one seed under a 60-step cap, from committed summaries. That agent took 0–3 whole-file reads in 44–60 actions (C6, log).

A DAgger-style relabel [4] run on 39 demonstrations, none read-required, moved the post-span reread 11/12 to 0/12 on held-out instances disjoint in seed and template, with mean input 1,157 to 748 and a matched-success saving of 1.59x; held-out pass moved 11/12 to 10/12, one rescue against two `xor` losses that pass visible and fail held-out, not distinguishable from noise at $n=12$ and one seed (C33).

3.3 Does the moment the diagnostic arrives change the outcome?

Accepted type-clean and held-out-correct outcomes were 1/12 under control, 10/12 with one-shot diagnostics at revision and 11/12 behind a submission gate, for task-bootstrap effects against control of +0.375 [+0.250, +0.500] and +0.417 [+0.292, +0.500]; after-every-edit stayed at 1/12 and +0.000 [−0.125, +0.125], on a channel that fired in 1 of 12 defect rows because the model edited before submitting in only 1 of 12 (C32).

Twelve seeded defect/clean pairs	Control	After every edit	One-shot	Gate
Bad completion accepted	11/12	11/12	2/12	1/12
Defect-cohort revision tokens	787	754	1,210	1,380
Defect-cohort wall time (s)	27.8	14.5	68.3	62.7
Clean-cohort revision tokens	591	—	619	591
Clean-cohort wall time (s)	9.4	9.9	9.6	9.5

After-every-edit fired in 1 of 12 defect rows; mean edits 0.08 against 0.17 in control.

On clean work the gate matched control in both tokens and wall time, and on defective work it spent about 2.2x control's wall time, turning 11/12 accepted bad completions into 1/12. The gate rejected 10 of 12 submissions, all 10 completing repair, retest and resubmission, and 2 of 12 self-repaired before-hand; on `auth_pipeline_handler` the model received the diagnostic and submitted anyway with zero edits, so the gate's one-task edge over revision came from refusal rather than information. Its single miss, `auth_shapes_protocol`, was type-clean after self-repair and failed held-out. The 0/12 false rejections are an

apparatus check, since each clean control is that defect’s validated gold and the checker is deterministic; population false-rejection rate is unmeasured. Fresh calibration produced no usable natural checker-detectable defect: frontier inference sat at 18/18 and a descriptive 27B authoring arm at 12/12, 0/3 natural 7B drafts were coherent, and 2/8 coherent 14B drafts were already type-clean, so these twelve defects were seeded (C10, C11, C16, C22). A separate execution-feedback grid (two frontier models, 14 tasks, three seeds, three delivery modes) passed 252/252 (C12).

4. Findings

4.1 Delivery

On a fifteen-task constructed dispatch suite at a single seed, one rollout per cell, semantic navigation added little over grep and ranged reads: a capable model resolves a dispatch-ambiguous target by reading the receiver’s type wherever it is visible and opening the right file directly, so the lookup delivers information the agent already holds. Cost stayed flat as the type moved from a call-site annotation to the test’s construction site to factory indirection, so resolution tracked the type wherever it remained readable. The load-bearing input is correct types in the source, rather than the server that queries them. The exception is a constructed regime that withheld the type-bearing source, where framed lookup resolved every rollout and text did not, over fifteen tasks at five seeds; the same operation left merely available changed nothing there. So delivery is not one lever but two, and the second is the one that matters: what a language server adds in this regime is contingent on the agent electing to use it, which is the same condition the retrieval results turn on. Sound types also sharpen the resolver mechanically, but that precision bought no agent-level outcome in a two-task pilot.

4.2 Form

A compact span is cheaper than a read only when it replaces the read, and whether it does is a property of the agent’s policy, not of the tool. The span is cheaper by a wide margin against whole-file retrieval and by a narrower one against a competent grep-plus-ranged-read interface. Election is separable: framing changed which operation the capable models called, and training changed it on a 7B. Substitution did not follow: on twelve instances per model at one seed, a pushed span was reread by a local 27B and, in two independent runs, by four frontier API models, and neither a sufficiency instruction nor self-election removed it. In a three-task, single-seed probe with a frontier model on sympy, astropy and sphinx, definition calls sat on top of reads, and the whole-file read that the largest ratios are measured against was rare, so the achievable regime there is the smaller one. Relabel training removed the reread where instruction and election could not, on one model at one seed; what it trained is blanket suppression of the post-span read, untested where the span is insufficient.

4.3 Timing

On twelve seeded defect/clean pairs, one model, one seed, a checker at the submission boundary converted accepted bad completions into repaired ones and one-shot delivery at revision performed comparably, while delivering the same checker after every edit changed nothing, on a channel that fired in one of twelve defect rows. Late delivery works; early delivery is unmeasured. The submission gate is preferable to delivery at revision because it charges its cost only to defective submissions: clean work cost no more than control in tokens or wall time, and the price fell on wall time when a submission was defective. A type-clean gate is only as behaviorally sound as its checker. Natural prevalence is unmeasured, and the same ceiling that forced seeding left execution feedback nothing to move on the tested small-task suite.

5. If you use a coding agent

On a typed Python codebase, the clearest return is a type check at the submission boundary with a repair-and-resubmit loop.

Lever	Do this	Changes if
Do nothing	Add no semantic navigation tool over typed source the agent already opens (C9, C30)	Your source hides receiver types
Add a tool or MCP server	Add one only where the receiver’s type is unreadable in source the agent opens: generated stubs, vendored or compiled boundaries. Installing it is not the intervention — an available but unframed operation changed nothing (C30, C27)	You measure targets the agent misses

Lever	Do this	Changes if
Skill or prompt change	Tell the agent to use the operation and when; framing is what moved resolution where the tool helped at all. Expect a change in tool choice, and verify post-span reads before budgeting a saving (C3, C30, C31, C34)	A prompt removes the post-span read
Wire a hook	Gate submission on the checker, requiring repair and retest before acceptance; after-every-edit firing is untested (C32)	Your checker covers behavior, or your model edits repeatedly
Fine-tune	Fine-tune only with weights you control, ruling out a hosted agent; it buys blanket suppression of the post-span read, not a conditional policy (C33)	Read-required boundaries are trained

Keep type annotations correct and present; the text baseline exploited exactly that. On your own workload, measure wrong-file edits, post-span reads of the defining file and spurious gate rejections; the clean-work cost here came from workspaces guaranteed clean. No result here ranks MCP servers, skill files and project instructions as delivery surfaces.

6. Future work

Every estimate rests on constructed workspaces: the closest real-repository candidate, one Django case with a working environment and genuine override ambiguity, was never audited for leakage or fix-site resolution (C17).

The hidden-type effect is one apparatus at one model: fifteen constructed tasks whose type is withheld by redaction rather than by the ordinary reasons real code hides a type, so whether it transfers to generated stubs, vendored packages or compiled boundaries is untested. A read-required boundary case, absent from the training set, would show whether blanket suppression can be made conditional. An in-loop design with repeated edits would exercise the after-every-edit channel. Rejection precision needs a gate run over drafts that are not the defect’s own gold.

Real-repository substitution frequency and live-server latency are unmeasured. Six four-seed files lack their seed-2/3 shards and no artifact records a server version. Static tooling does not reach dynamic behavior, incorrect annotations, Any-heavy boundaries or logic outside the checker. The resolution answer holds only while the agent can read the type-bearing source cheaply, and every workspace tested was small enough that it could.

7. Conclusion

Semantic navigation added little for a capable agent on the dispatch suite tested, because the receiver’s type was readable in the source the agent opened; where the type was withheld it helped, but only once the agent was framed to reach for it. A compact definition span costs less than a read only when it displaces one; only training produced that displacement, and pushing, instructing and self-election did not. A type check at the submission boundary, with an explicit repair-and-resubmit loop, converted accepted bad completions into repaired ones on twelve seeded defect pairs.

References

- [1] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. Proceedings of the ACM on Programming Languages, 8(OOPSLA2), 2024. arXiv:2409.00921.
- [2] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: Lsp-guided rag for language-agnostic real-time unit test generation. arXiv preprint arXiv:2510.22210, 2025.
- [3] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting llm-based repository-level code completion with static analysis. arXiv preprint arXiv:2406.10018, 2024.

- [4] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 2011. PMLR v15:627–635.